

# CSC 718: Operating Systems & Parallel Programming

## Assignment 1

### Multithreading: Queue + Workers (pthreads)

#### Overview

In this assignment, you will build a small but realistic multithreaded system in C or C++ using POSIX threads (pthreads). The emphasis is on correctness, synchronization, and performance awareness.

The assignment has three progressive requirements:

- **R1:** Implement a correct bounded queue with synchronization.
- **R2:** Build a worker pool that executes jobs from the queue.
- **R3:** Run scaling experiments and analyze performance.

#### Learning Goals

By completing this assignment, you will be able to:

- Use pthread mutexes and condition variables correctly without busy-waiting.
- Design and test a bounded producer-consumer queue.
- Implement a worker pool with a clean shutdown mechanism.
- Measure execution time and analyze parallel scaling.
- Identify synchronization and workload characteristics that affect performance.

#### Project Description

You will implement a system in which multiple worker threads pull jobs from a bounded queue. The main thread submits jobs, waits for all jobs to complete, and then shuts down the workers cleanly.

Jobs should be computationally simple but non-trivial. Examples include:

- Counting primes up to a specified value.
- Hashing or processing a fixed-size buffer.
- Executing a loop containing a fixed amount of computation.

Keep job costs reasonably uniform and deterministic so that scaling results are meaningful.

**Performance workload requirement:** Choose the number of jobs and the job workload so that the 1-worker execution takes at least a few seconds. Very short runs do not provide reliable timing or scaling results.

#### R1: Bounded Queue (40 pts)

Implement a thread-safe bounded queue.

#### Requirements

- Implement blocking enqueue() and dequeue() operations.
- Use exactly one mutex and two condition variables, such as not\_full and not\_empty.
- Do not use busy-waiting or polling.
- Write a unit test using multiple producers and consumers.
- Verify that every submitted job is processed exactly once.
- Include a simple correctness check, such as comparing the expected and observed sum of job IDs.

## Deliverables

- queue.h and queue.c, or corresponding C++ files, containing the queue API and implementation.
- r1\_test.c, or corresponding C++ file, containing the test program.

## R2: Worker Pool (40 pts)

Build a fixed-size worker pool using the bounded queue from R1.

### Requirements

- Create a fixed number of worker threads.
- The main thread must enqueue M jobs and wait until all jobs have completed.
- Workers must repeatedly dequeue and execute jobs.
- Workers must terminate cleanly when shutdown is requested.
- You may use poison-pill jobs, a shutdown flag with condition-variable notification, or another correct shutdown mechanism.
- The program must not terminate with blocked threads, unfinished jobs, or resource leaks.

At the end of execution, print a clear summary containing:

- Number of worker threads.
- Number of jobs processed.
- Total elapsed time.
- Throughput in jobs per second.

**Timing requirement:** Start the timer after the worker threads have been created and immediately before job submission begins. Stop the timer after all jobs have completed and before worker-pool shutdown. Use the same timing region for all performance experiments.

## Deliverables

- pool.h and pool.c, or corresponding C++ files, containing the worker-pool implementation.
- r2\_main.c, or corresponding C++ file, containing the main driver.

## R3: Scaling Study and Report (20 pts)

Evaluate how the worker pool performs as the number of worker threads increases.

Run the program with **N in {1, 2, 4, 8} workers**.

Use the same conditions across all four configurations:

- Number of jobs (M).
- Job type and workload.
- Input data.
- Timing method.
- Computing environment.

## Report

For each value of N, report:

- Wall-clock execution time.
- Speedup:  $S(N) = T(1) / T(N)$ .
- Evidence of execution, such as screenshots or console output showing the worker count and timing.
- A short discussion of the observed scaling behavior.

Your discussion should identify likely performance limitations where applicable. Examples include:

- Queue contention.
- Synchronization overhead.
- Job granularity.
- Thread-management overhead.
- Limited available CPU cores.
- Other shared-resource contention.

Do not assume that increasing the number of workers must always improve performance. Interpret the results you actually observe.

## Deliverable

Submit r3\_results.pdf containing:

- A table of results.
- Speedup calculations.
- Execution evidence.
- A short technical discussion of the results.

Keep the discussion concise. The written discussion should not exceed one page.

## Technical Constraints

- Use C or C++.
- You must use the POSIX threads API.
- Required pthread functionality includes appropriate use of functions and types such as pthread\_create, pthread\_mutex\_t, and pthread\_cond\_t.
- Do not use std::thread, std::mutex, or std::condition\_variable.
- Use a monotonic or steady timer, such as clock\_gettime(CLOCK\_MONOTONIC, ...) or std::chrono::steady\_clock.
- Keep inputs deterministic. Use a fixed random seed or a fixed job set where applicable.

## Submission Requirements

Upload a single ZIP file to D2L containing the following.

### Source Code

- Queue implementation.
- Worker-pool implementation.
- R1 test program.
- R2 main driver.
- Any additional source or header files required to build the program.

### README

Include:

- Exact build instructions.
- Exact run instructions.
- Worker-count parameter (N).
- Number of jobs (M).
- Job type and workload parameters.

- Brief description of the timing method.
- Any assumptions or known limitations.

### R3 Report

Include r3\_results.pdf with:

- Timing table.
- Speedup calculations.
- Execution evidence.
- Short performance discussion.

### AI Use

Limited use of generative AI tools is permitted for activities such as clarifying syntax, exploring approaches, debugging, or other technical support.

AI tools must not be used to generate substantial portions of the submitted code, performance analysis, report text, results, screenshots, logs, or other execution evidence.

If you use an AI tool, include a brief AI Use Statement in your README. Identify the tool used and briefly describe how it assisted you. If no AI tools were used, state this explicitly.

You are responsible for understanding, testing, and verifying all submitted work.

## Grading Rubric (100 pts)

### R1: Bounded Queue (40 pts)

- Correct mutex and condition-variable synchronization; no races or deadlocks: 15 pts
- enqueue() blocks when full and dequeue() blocks when empty: 10 pts
- Unit test demonstrates exact-once processing across multiple threads: 10 pts
- Modular design, readability, and appropriate comments: 5 pts

### R2: Worker Pool (40 pts)

- Workers correctly dequeue and execute jobs; all jobs complete: 15 pts
- Clean shutdown with no hangs, blocked threads, or resource leaks: 10 pts
- Driver executes correctly and reports jobs, elapsed time, and throughput: 10 pts
- Code organization and readability: 5 pts

### R3: Scaling Study and Report (20 pts)

- Complete timing results for  $N = 1, 2, 4, 8$ : 5 pts
- Correct speedup calculations: 5 pts
- Clear execution evidence: 5 pts
- Concise and technically sound analysis of scaling and bottlenecks: 5 pts